

(Java) תכנות אינטרנט ב – תרגיל 1

Concurrent WebCrawler & Content Analyzer
Java OOP, Threads, HTTP, Design Patterns

נביאים: <https://classroom.github.com/a/QVsSPEN1>

שטראוס: <https://classroom.github.com/a/h3JMOLq4>

סקירה כללית

בתרגיל זה, תיישמו WebCrawler שמעבד כתובות URL באופן הדרגתי, מגלה קישורים חדשים במהלך הביצוע, מיישם אסטרטגיות ניתוח ומפיק דוח מובנה.

מטלה זו מדגישה תחום סנכרון, בטיחות הליכים ועיצוב תכנה. פתרונות טריוויאליים סדרתיים או מבוססי batch לא יקבלו את מלוא הנקודות.

סביבת עבודה

Java JDK 25.

כלי בנייה: Maven. קובץ pom.xml חייב להיות נוכח. כל התלויות חייבות להיות מוצהרות שם; הוספת קבצי JAR ידנית אסורה.

ספריות מומלצות:

- **Jsoup 1.17+** ([org.jsoup:jsoup](https://jsoup.org/)) – HTML parsing, visible-text extraction, and link discovery.
- **Jackson Databind 2.17+** (com.fasterxml.jackson.core:jackson-databind) – JSON serialization.

אין להשתמש בספריות צד-שלישי נוספות ללא אישור מראש.

הרצת התוכנית

מחלקה ראשית חייבת להיקרא Main

פורמט הרצה:

```
java Main --analysis <strategies> # WORD_COUNT,BROKEN_LINKS,... (required)
--poolsize <n> # thread-pool size > 0 (required)
--depth <d> # max crawl depth ≥ 0 (required)
--input <file> # seed URL file (required)
--output <file> # report destination (required)
```

for example (if compiled into a JAR file):

```
java -jar target/ex1-solution-1.0-SNAPSHOT.jar \
--analysis WORD_COUNT,BROKEN_LINKS,KEYWORD_FREQUENCY \
--poolsize 3 --depth 2 \
--input seeds.txt --output report.json
```

ארגומנטים

1. <analysis> – רשימה מופרדת בפסיקים של סוגי ניתוח מתוך – WORD_COUNT, MOST_LINKED_DOMAIN, BROKEN_LINKS, KEYWORD_FREQUENCY

2. מספר שלם חיובי שאינו אפס – `<poolsize>`.
3. מספר שלם אי-שלילי – `<depth>`.
- עומק 0 פירושו "רק דף זה, ללא צאצאים"; עומק 1 פירושו "דף זה וצאצאים ישירים" וכו'.
4. `<inputFile>` – קובץ טקסט המכיל כתובת URL אחת בכל שורה.
5. `<outputFile>` – קובץ json המכיל תוצאות זחלן

מודל הסריקה

1. הסריקה חייבת להיות פרוגרסיבית. כתובות URL מתגלות ומתוזמנות במהלך הביצוע.
2. אסור לעבד כתובת URL יותר מפעם אחת.
3. ההחלטה האם כתובת URL כבר נבדקה חייבת להתבצע באופן אטומי ובאופן בטוח (thread safe) בזמן הגילוי.
4. אסור לטעון מראש או לחשב את גרף הסריקה המלא לפני העיבוד.
5. סריקה וניתוח חייבים להתרחש בו זמנית.
6. רק כתובות URL מסוג HTTP/HTTPS מותרות.
7. סדר הפלט של כתובות URL שהתגלו חייב לפעול לפי סדר הגילוי הראשון. הערה: שמירה על סדר זה תחת מקביליות היא כשלעצמה אתגר סנכרון..
8. הסורק חייב לכבד את העומק בקפדנות.
9. חסימת כל הליכי ה-worker בזמן המתנה להשלמה גלובלית אסורה, כגון חסימת כל הליכי ה-worker על מחסום או נעילה משותפים כך שאף ליכי ה-worker לא יוכל להתקדם.

טיפול בשגיאות

- Malformed URL: print '`<url> malformed`' to System.err
- Unreachable URL: print '`<url> failed`' to System.err and continue
- Output file creation failure: print '`error saving report`'
- Input file failure: print '`invalid input file`' and exit
- Pool size ≤ 0 : print '`invalid pool size`' and exit
- Max depth < 0 : print '`invalid depth`' and exit
- Analysis errors:
 - Unknown analysis: print '`<name> is unknown`' and ignore it.
 - If no valid analysis remains: print '`no valid analysis`' and exit.

כללי עיבוד עמודים

רק דפים המחזירים HTTP 200 מנותחים.

- HTTP (3xx) עוקבות אחר הפניות באופן אוטומטי, עד למקסימום של 20 קפיצות (ברירת המחדל של Jsoup). קוד הסטטוס הסופי לאחר כל ההפניות הוא זה המשמש לניתוח וזיהוי קישורים שבורים. השתמשו ב-`HttpURLConnection.setFollowRedirects(true)` או במעקב הפניות מובנה של Jsoup
- פסקי זמן (Timeouts): מגדירים פסק זמן לחיבור ולקריאה של 3 שניות בכל בקשת HTTP. כתובות URL שחורגות ממגבלות אלה מטופלות כבלתי ניתנות להשגה ויש לדווח עליהן כ-'`<url> failed`' ל-System.out

יש לחלץ מכל url את:

- Title
- Word count (visible text only)
- Number of outgoing links
- Domain name
- HTTP status code

הגדרות ניתוח

- WORD_COUNT : ספירת מילים כוללת בכל הדפים.
- MOST_LINKED_DOMAIN : הדומיין עם ספירת הקישורים היוצאים הגבוהה ביותר (שובר שוויון לקסיקוגרפי)
- BROKEN_LINKS : רשימת כתובות URL המחזירות סטטוס != 200 (בסדר גילוי)
- KEYWORD_FREQUENCY : מילות מפתח מוגדרות מראש כ: java, thread, pattern case) (insensitive).

פלט JSON

The output must be a single JSON object with two top-level keys:

- **pages** – array of page objects in first-discovery order. Each object contains: url, title, wordCount, outgoingLinks, domain, status, depth.
 - **analysis** – object containing only the analyses that were requested.
- KEYWORD_FREQUENCY is a nested object mapping each keyword to its count.

For demo purpose we use this seed example using all 4 analysis and depth 0:

```
https://en.wikipedia.org/wiki/Java_(programming_language)
https://httpbin.org/status/404
https://httpbin.org/status/500
https://example.com
```

the required output structure (all four analyses requested, one page crawled) is:

```
{
  "pages" : [
    {
      "url" : "https://en.wikipedia.org/wiki/Java_(programming_language)",
      "title" : "Java (programming language) - Wikipedia",
      "wordCount" : 9000,
      "outgoingLinks" : 1717,
      "domain" : "en.wikipedia.org",
      "status" : 200,
      "depth" : 0
    },
    {
      "url" : "https://httpbin.org/status/404",
      "title" : "",
      "wordCount" : 0,
      "outgoingLinks" : 0,
      "domain" : "httpbin.org",
      "status" : 404,
      "depth" : 0
    },
    {
      "url" : "https://httpbin.org/status/500",
      "title" : "",

```

```
    "wordCount" : 0,
    "outgoingLinks" : 0,
    "domain" : "httpbin.org",
    "status" : 500,
    "depth" : 0
  },
  {
    "url" : "https://example.com",
    "title" : "Example Domain",
    "wordCount" : 19,
    "outgoingLinks" : 1,
    "domain" : "example.com",
    "status" : 200,
    "depth" : 0
  }
],
"analysis" : {
  "WORD_COUNT" : 9019,
  "BROKEN_LINKS" : [ "https://httpbin.org/status/404", "https://httpbin.org/status/500"
],
  "MOST_LINKED_DOMAIN" : "en.wikipedia.org",
  "KEYWORD_FREQUENCY" : {
    "java" : 386,
    "thread" : 0,
    "pattern" : 0
  }
}
}
```

Pages with a non-200 status are included in the `pages` array (with their actual status code; title, wordCount, and outgoingLinks may be null or 0).

If `BROKEN_LINKS` was requested but no broken URLs exist, output `"BROKEN_LINKS": []`.

הערכת סנכרון

קריטריונים לבדיקה:

- Correct atomic visited-check logic
- Proper synchronization of shared data
- Absence of race conditions
- Minimal necessary locking
- Avoidance of global synchronized blocks
- No busy waiting
- No sequential fallback implementation

דרישות עיצוב

עליכם להשתמש לפחות ב-4 תבניות עיצוב.

תבניות שכדאי לבדוק (אך לא רק):

Strategy, Factory, Observer, Singleton, Chain of Responsibility, Decorator, Visitor, Template Method, Builder.

ירושה פשוטה לבדה אינה מספיקה. המערכת חייבת להיות פתוחה להרחבה וסגורה לשינוי (OCP)

דרישת הרחבה

דרישות הרחבה מעידות על עיצוב תואם לדרישות עיצוב כלומר יכולת להרחיב את התכנית ללא שינויים בקוד הרלוונטי.

ממשו לפחות 2 מהבאים:

1. סוג ניתוח נוסף

2. פורמט פלט חלופי ('וכו html, csv, text')
הוסיפו בפורמט הרצה :

```
--format json # output format, default json (optional)
```

3. סינון דומיין : allow only specific domains
הוסיפו בפורמט הרצה :

```
--domains <d1,d2,...> # hostname whitelist (optional)
```

4. מקור קלט חלופי כגון standard input (console)

דרישות בדיקה

יש להגיש קובץ קלט לדוגמא בשם seeds.txt

בדקו עם:

- poolSize = 1
- poolSize > 1
- depth = 0
- depth > 1
- URL כתובות כפולות
- קישורים מעגליים
- לפחות 100 עמודים בסך הכל

קריטריונים להערכה

- נכונות – 30%
- משמעת סנכרון – 25%
- איכות עיצוב ותבניות – 20%
- יכולת הרחבה 15% –
- איכות קוד ותייעוד – 10%

דרישות הגשה ל-github:

README.md with:

1. emails of all students (edu.jmc.ac.il) critical for automatic grading and reporting in moodle.
A missing or wrong email will break the process and be penalized -5
2. how to compile: Precise instructions for compilation
3. how to run: Precise instructions for execution
4. List of design patterns used

5. List of implemented extensions

- Continuous Git history with meaningful commits
- (at least one) Branching/merging (do not delete branches before submission)

ובדקו את הסילבוס

בהצלחה!